

שבוע שנים עשר על עמוד אחד

שבוע: 12

שני המשפטים של השבוע

טיפוסים מתארים צורה, לא כוונה – ו- `as` הוא הצהרה, לא בדיקה.
ארגון מחדש הוא שינוי שאינו משנה התנהגות. אם ההתנהגות השתנתה, לא ארגנת – שינית.

מה נמחק בקומפילציה, ומה לא

מה שכתבת	קיים בזמן ריצה?
<code>interface</code>	לא
<code>type</code>	לא
<code>string</code> : על פרמטר	לא
<code>as X</code>	לא
<code><T></code> בגנריקה	לא
<code>function isX(...)</code>	כן
<code>instanceof</code>	כן
<code>typeof</code>	כן

כל מה שבחצי העליון נעלם. לכן מה שיוצא מ-`localStorage` ומהרשת נבדק בחצי התחתון, ולא מתואר בעליון.

ארבע דרכים לומר "זה מסוג X", ומה כל אחת שווה

מה כותבים	מה זה אומר	מה רץ
<code>: any</code>	תפסיק לבדוק, כאן ובהמשך	כלום
<code>: unknown</code>	אף אחד עוד לא בדק	כלום – אבל אסור לגעת עד שמישהו יבדוק
<code>as T</code>	אני מבטיח שזה T	כלום
<code>value is T</code>	הנה ההוכחה	גוף הפונקציה

```
let payload: unknown;
payload = await res.json();
return toSuggestions(payload);
```

`res.json()` מוגדר בטיפוסי ה-DOM כמחזיר `Promise<any>` — לא `unknown`. בלי ההצהרה המפורשת, גישה לכל שדה בתשובה עוברת את המהדר בשקט, גם על מסמך שגיאה של 404.

שגיאות המהדר שתפגוש היום, ומה הן אומרות

ההודעה	מה לעשות
Property 'x' does not exist on type 'unknown'	בדיוק מה שביקשת. צריך שומר לפני הגישה
'x' is possibly 'null'	הוא באמת עשוי להיות. פסוק שמירה, או <code>()\$</code>
NodeListOf<Element> must have a '[Symbol.iterator]'	חסר "DOM.Iterable" ב- <code>lib</code> . כבר שם אצלך
Type 'Date' is not assignable to type 'string'	זה הבאג של שבוע 10, שנתפס לפני ההרצה
'error' is of type 'unknown' בתוך <code>catch</code>	<code>error instanceof Error</code> לפני <code>error.name</code>
Cannot assign to 'x' because it is a read-only property	<code>readonly</code> תפס מוטציה. החלף במקום לשנות

ארבעים שגיאות אינן ארבעים בעיות. תקן את הראשונה בלבד, הרץ שוב, וספור.

שלוש הטעויות שכלי AI עושה כאן, כל פעם

- `catch (error: any)` — התיקון הראשון שהוא מציע תחת `strict`, והוא מכבה בדיוק את מה שביקשת. הנכון: `error instanceof Error`.
- `(await res.json()) as X` — מהדר, קצר יותר, בודק בדיוק כלום.
- `async function getJson<T>(url): Promise<T>` — נראה מקצועי, ואין `T` בזמן ריצה. הגרסה הכנה מקבלת שומר כפרמטר.

הקטלוג של ארגון מחדש – שבעה מהלכים

המהלך	הריח שקורא לו
חילוץ פונקציה	הערה שמסבירה מה חמש השורות הבאות עושות
שם שאומר כוונה	<code>handleClick2 · temp · data</code>

המהלך	הריח שקורא לו
הסרת כפילות	אותן שלוש שורות פעמיים, עם מילה שונה
פסוקי שמירה	העבודה האמיתית ארבע הזחות מהשוליים
פיצול לפי אחריות	פונקציה ששמה צריך את המילה "ו"
שם לערך קסום	86400000 · '-' · 0.6
פישוט תנאי	if (x === true) return true; else return false;

אתה לא צריך לזכור את הטבלה. אתה צריך לזהות את העמודה הימנית בקוד שלך.

איך מארגנים מחדש בבטחה כשאינ בדיקות

- מהלך אחד בכל פעם. חילוץ, או שינוי שם — לא שניהם.
- בדיקה אחרי כל אחד. רענון, לחיצה, השוואה למסך. עשר שניות.
- קומיט אחרי כל אחד. ככה מגלים איזה מהלך שבר.
- רשום מה אסור שישתנה לפני שאתה מתחיל — שלוש התנהגויות, ואיך תבדוק כל אחת.
- אם אי אפשר לבדוק, אל תעשה. צמצם את ההיקף עד שאפשר.

חמשת הקריטריונים לארגון מחדש טוב

#	הקריטריון	מי עונה
1	ההתנהגות זזה — כל קלט, כל פלט	מכונה
2	מה שהיה כתוב פעמיים כתוב עכשיו פעם אחת	מכונה
3	השמות אומרים מה, לא איך ולא איפה	אדם
4	השינוי בגודל המשימה, ולא גדול ממנה	אדם
5	שינוי עתידי שנגע ב-N מקומות נוגע עכשיו באחד	אדם

אלה חמשת הקריטריונים של המבחן, והם לא סוד.

שמונה ההדגמות של השיעור

פתח כל אחת מהן ב-Live Server מתוך `demos/`. כל הודעת מהדר שמופיעה בהן היא פלט אמיתי שנלכד מהרצה על הקבצים `demos/tsc-cases/` ב-.

התיקיה	מה היא מודדת
01-the-bug-types-catch	הבאג של שבוע 10, חי, ליד השורה שהייתה מונעת אותו

התיקיה	מה היא מודדת
02-erasure	אותו מודול לפני ואחרי tsc . ספור מה נשאר
03-any-vs-unknown	אותו גוף תשובה, שני קוראים
04-narrowing	ארבע בדיקות מול שישה ערכים. קרא את שורת null
05-generic-trap	שתי חתימות, אפס שגיאות מהדר, ותוצאה אחת שגויה
06-dom-cast	ארבע שגיאות אמיתיות, ושתי הודעות ריצה שאפשר להשוות
07-behaviour-preserving	ארבע גרסאות, 240 קלטים, שאלה אחת
08-graded-refactor	ארבע הגשות, מנוקדות בשידור חי. דרג אותן לפני שתלחץ

היום בתרגול

כמה זמן	מה
40 דקות	מטלת כיתה 12 — העברת שכבת הרשת לטיפוסים. מפרט, כלי, ביקורת
40 דקות	ניתוח מלא — לבד, בלי AI, בלי חיפוש, בתנאי מבחן
10 דקות	הליכה על התשובות, והבודק האוטומטי רץ על המסך

והפרויקט מוגש בסוף השבוע

```
node .checks/run-checks.mjs
```

מתוך `project/` . **זו בדיקת אותה בדיקה ש-CI מריץ, רק לפני שדחפת.** אחר כך עבור על `SUBMISSION_CHECKLIST-he.pdf` — חמישה דברים שאף מכונה לא יכולה לבדוק, ואתה יכול, בעשר דקות. **הגשה במודל: כתובת המאגר + ה-SHA של הקומיט.**

שלוש שאלות לפני שאתה יוצא

1. כתבת `interface Item` וקימפלת. **מה נמצא בקובץ הפלט?**
 2. `res.json()` החזיר מסמך שגיאה של 404. **מי מבין `any`, `unknown` ו-`as` אמר לך?**
 3. העברת קובץ אחד מתוך שישה. **כמה מהם נבדקים עכשיו על ידי המהדר?**
- התשובות:** (1) שום דבר מהטיפוס — רק השומר. (2) אף אחד; `unknown` רק מנע ממך לגעת עד שבדקת, וזה לא אותו דבר. (3) **שישה** — ה-`d.ts`. נותן את הטיפוסים גם למי שנשאר JavaScript, וזו כל הסיבה שהעברה הדרגתית עובדת.